



UNIVERSIDADE DO OESTE DE SANTA CATARINA – UNOESC
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
PRÁTICA EXTENSIONISTA IV

RELATÓRIO DE PRÁTICA EXTENSIONISTA IV

Conectando Comunidades: Sistema de Empréstimo de Ferramentas e Serviços

Integrantes do grupo:

Carine Renostro

Gabriel Giacobbo

Laurence Alves Ribeiro

Tais Vitória Ribeiro dos Santos

Thuaina Alexandra Maia

Yuri Aguiar Urbano

Joaçaba, 2026

SUMÁRIO

1 CONTEXTUALIZAÇÃO, CARACTERIZAÇÃO DO PROBLEMA E SOLUÇÃO PROPOSTA.....	3
2 REPOSITÓRIO E DOCUMENTAÇÃO NO GITHUB.....	4
3 DIAGRAMA DE PACOTES (ARQUITETURA DA APLICAÇÃO).....	5
4 DIAGRAMA DE ARQUITETURA DE IMPLANTAÇÃO.....	6
5 DIAGRAMA DE ARQUITETURA DEVOPS.....	7
6 DESCRIÇÃO E JUSTIFICATIVA DA INFRAESTRUTURA DE DEPLOY.....	8
6.1 Considerações iniciais.....	8
6.2 Hospedagem: máquina virtual em cloud IaaS.....	9
6.2.1 Plataformas como serviço (PaaS).....	10
6.2.2 Arquiteturas serverless.....	11
6.2.3 Servidor próprio (self-host).....	11
6.3 Exposição pública: Cloudflare Tunnel.....	11
6.4 Banco de dados: PostgreSQL em container Docker.....	12
6.5 Gerenciamento de processos: systemd.....	12
6.6 Integração e entrega contínuas: GitHub Actions.....	13
6.7 Ativação das contas e da estrutura.....	13
6.8 Riscos e limitações conhecidos.....	13
6.9 Síntese da justificativa.....	14
REFERÊNCIAS.....	15
APÊNDICE A - DIAGRAMA DE PACOTES UML (ARQUITETURA DA APLICAÇÃO).....	17
APÊNDICE B - DIAGRAMA DE IMPLANTAÇÃO UML.....	18
APÊNDICE C - DIAGRAMA DE ARQUITETURA DEVOPS.....	19

1 CONTEXTUALIZAÇÃO, CARACTERIZAÇÃO DO PROBLEMA E SOLUÇÃO PROPOSTA

O modelo de consumo contemporâneo é caracterizado pela aquisição individual de bens de uso esporádico, como ferramentas de manutenção e utensílios domésticos de alto custo. Esse comportamento gera um acúmulo de itens subutilizados nas residências, resultando em desperdício de recursos financeiros e aumento do impacto ambiental decorrente da produção excessiva e do descarte precoce. Embora o interesse por práticas de "faça você mesmo" (DIY) tenha crescido (ALLIED MARKET RESEARCH, 2022), a posse individual permanece como a regra, ignorando o potencial de aproveitamento de recursos já existentes na própria comunidade.

Apesar da disposição de muitos indivíduos em compartilhar esses recursos, a ausência de mecanismos formais e seguros impede que o consumo colaborativo se consolide. Atualmente, o empréstimo entre vizinhos ocorre de forma informal e desestruturada, o que gera insegurança quanto à integridade dos objetos e incerteza sobre os prazos de devolução. Essa falta de controle e de um canal de comunicação eficiente desgasta as relações comunitárias e desestimula a colaboração, perpetuando o ciclo de consumo linear e a dependência de novas compras para necessidades temporárias.

Diante desse cenário, o projeto Trocaí surge como uma solução tecnológica projetada para intermediar e formalizar o compartilhamento local de materiais e ferramentas. A plataforma utiliza a modelagem de sistemas para estruturar processos de empréstimo e cadastro de itens, garantindo que cada transação seja registrada e monitorada. Ao integrar funcionalidades de geolocalização e verificação de identidade, o sistema reduz as barreiras logísticas e mitiga os riscos de segurança, transformando a confiança interpessoal em um processo sistêmico e auditável.

A solução proposta alinha-se aos Objetivos de Desenvolvimento Sustentável da ONU, em particular ao ODS 12, que trata de consumo e produção responsáveis (ONU, 2015; PNUD, [s.d.]), promovendo a economia circular através da otimização da vida útil dos produtos. Mais do que uma ferramenta de gestão de inventário, o sistema visa fortalecer os vínculos comunitários e democratizar o acesso a equipamentos de qualidade sem a necessidade de aquisição. Dessa forma, a plataforma converte a subutilização de bens em valor social e ambiental, oferecendo uma alternativa viável, segura e escalável ao modelo de consumo tradicional.

2 REPOSITÓRIO E DOCUMENTAÇÃO NO GITHUB

O código-fonte, a documentação e os diagramas do projeto estão publicados no repositório público do GitHub, disponível em: <https://github.com/g8148/Trocai>.

O repositório adota a estrutura de monorepositório, reunindo backend, frontend, infraestrutura, documentação e automação em um único local. Essa organização facilita o versionamento conjunto das camadas da aplicação e permite que um único workflow de integração e entrega contínuas publique todo o sistema. O Quadro 1 descreve a função de cada diretório e arquivo principal.

Quadro 1 — Estrutura do repositório do projeto

Caminho	Conteúdo
README.md	Visão geral do projeto, stack tecnológica, arquitetura, estrutura do repositório, execução local, autenticação, apps do backend e resumo do deploy, com link para o infra/README.md
PLANO_PROJETO.md	Plano do projeto: arquitetura, modelos de dados, organização da equipe e cronograma
app/api/	Backend em Django 6 e Django REST Framework (core, apps de domínio e testes automatizados) e docker-compose.yml do PostgreSQL
app/web/	Frontend em Next.js 16.1.6, com dependências gerenciadas pelo npm (package-lock.json)
infra/README.md	Guia de provisionamento da VPS do zero, decisões de infraestrutura e limitações conhecidas do deploy
infra/deploy.sh	Script executado na VPS a cada deploy: instala dependências, faz backup do banco, aplica migrações, compila o frontend e reinicia os serviços
infra/systemd/	Units do systemd dos serviços trocai-api e trocai-web
infra/cloudflared/	Configuração de exemplo do Cloudflare Tunnel
doc/	Documentação das entregas do projeto, organizada por entrega: relatórios e imagens dos diagramas (pacotes, implantação e DevOps)
.github/workflows/deploy.yml	Workflow do GitHub Actions que realiza o deploy automático na VPS e a verificação de saúde dos serviços

Fonte: Elaboração própria (2026).

A documentação é mantida em Markdown, de modo que é renderizada diretamente pela interface do GitHub. O infra/README.md concentra a documentação mais técnica do projeto: descreve como provisionar a VPS e registra as limitações conhecidas do processo de deploy, retomadas nas Seções 5 e 6. Os segredos de implantação (endereço, usuário e chave SSH da VPS) não são versionados: permanecem cadastrados como GitHub Secrets, e as variáveis de ambiente locais seguem o modelo do arquivo .env.example.

3 DIAGRAMA DE PACOTES (ARQUITETURA DA APLICAÇÃO)

O diagrama de pacotes UML (Figura 1, Apêndice A) representa a organização lógica da aplicação Trocaí, que segue o padrão arquitetural API-First: o backend expõe uma API REST independente, consumida pelo frontend. O sistema está dividido em dois pacotes principais, web e api, que correspondem, respectivamente, aos diretórios app/web e app/api do repositório. Todas as setas tracejadas do diagrama são dependências com o estereótipo «use», indicando que o pacote de origem utiliza modelos ou serviços do pacote de destino, conforme a legenda da figura.

O pacote web, desenvolvido em Next.js 16.1.6, agrupa as páginas e rotas da aplicação (app), os componentes reutilizáveis de interface (components) e os serviços de acesso à API (lib), responsáveis pelas requisições HTTP e pelo armazenamento e renovação dos tokens JWT. A comunicação com o backend é representada pela dependência entre o pacote lib e o pacote core, realizada por meio de requisições REST/JSON com o token JWT enviado no cabeçalho Authorization.

O pacote api, desenvolvido em Django 6 com Django REST Framework, é composto pelo pacote core, que concentra as configurações do projeto, o roteamento raiz, a autenticação por JWT e a documentação OpenAPI, e pelo pacote apps, que reúne os sete módulos de domínio. As dependências entre esses módulos correspondem às chaves estrangeiras e às regras de negócio presentes no código:

- **accounts** é a base do domínio: o modelo User é referenciado por todos os demais módulos, e cada um deles depende de accounts;
- **items** depende de accounts, que identifica o proprietário do item;
- **loans** depende de items, de accounts e de notifications, este último utilizado para gerar os avisos de solicitação, aprovação e devolução;
- **reviews** depende de loans, pois cada avaliação está vinculada a um empréstimo, e de accounts, por meio dos campos reviewer (autor) e reviewed_user (usuário avaliado);
- **reports** depende de items, de loans e de accounts, que são os alvos possíveis de uma denúncia (campo target_loan, por exemplo, referencia o empréstimo denunciado);
- **chat** depende de accounts, que identifica os participantes, e de items, pois cada conversa pode estar ligada a um item;
- **notifications** depende de accounts, destinatário de cada notificação.

As dependências apontam sempre dos módulos mais específicos para os mais genéricos, sem ciclos entre pacotes, o que reduz o acoplamento e permite evoluir cada módulo

de forma independente. O pacote Bibliotecas externas, com estereótipo «external», agrupa os frameworks e bibliotecas de terceiros utilizados por cada camada.

4 DIAGRAMA DE ARQUITETURA DE IMPLANTAÇÃO

O diagrama de implantação UML (Figura 2, Apêndice B) descreve a distribuição física dos artefatos da aplicação em ambiente de produção. Os nós são diferenciados por estereótipo: «device» representa hardware físico (o dispositivo do usuário); «node» representa a máquina virtual contratada no Oracle Cloud; «executionEnvironment» representa ambientes de software que hospedam outros elementos (sistema operacional, systemd, serviços, Docker e container); e «external system» representa serviços de terceiros acessados pela internet (GitHub, Cloudflare e OpenStreetMap/Nominatim). As linhas contínuas são caminhos de comunicação e ligam sempre nós, com o protocolo e a porta indicados; os artefatos aparecem apenas dentro dos nós em que estão implantados.

Toda a solução é executada em uma máquina virtual Ampere A1 da camada Always Free do Oracle Cloud, com 2 OCPUs de arquitetura ARM e 12 GB de memória RAM, executando o Ubuntu 24.04 LTS. Dentro do sistema operacional, o systemd atua como ambiente de execução de três serviços: trocai-web.service, que executa o build de produção do Next.js 16.1.6 na porta 3002; trocai-api.service, que executa o Django por meio do servidor WSGI Gunicorn, com três workers, na porta 8000; e cloudflared.service, agente do Cloudflare Tunnel.

O banco de dados PostgreSQL 18 é executado no container trocai-db, gerenciado pelo Docker Engine. O volume postgres_data foi representado fora do container, dentro do Docker Engine, com uma dependência «mount»: o volume é gerido pelo Docker no sistema de arquivos do host e sobrevive à remoção ou recriação do container, que é justamente o motivo de utilizá-lo. Os arquivos enviados pelos usuários, os arquivos estáticos, os logs do Gunicorn e os backups do banco são artefatos armazenados diretamente no sistema de arquivos da VPS.

Todos os serviços escutam exclusivamente no endereço 127.0.0.1. O tráfego do usuário parte do navegador, chega ao Cloudflare por HTTPS, onde ocorrem a resolução de DNS e a terminação TLS, e é encaminhado à VPS pelo túnel estabelecido pelo cloudflared, conexão iniciada de dentro da VPS para fora. O cloudflared distribui as requisições conforme o hostname: trocai.<domínio> para o Next.js e api-trocai.<domínio> para o Django. O Django, por sua vez, consulta o OpenStreetMap/Nominatim por HTTPS para geocodificar os endereços dos usuários. O runner do GitHub Actions acessa a VPS via SSH na porta 22, única

porta de entrada liberada no firewall da instância, para atualizar o código e executar o script de deploy.

Cabe registrar que a VPS é compartilhada com outros projetos da equipe, conforme documentado no `infra/README.md`. Por essa razão, o PostgreSQL do Trocaí é publicado na porta 7510, já que a 5432 pertence ao banco de outro projeto; o Cloudflare Tunnel é compartilhado, com regras de ingress próprias para os hostnames do Trocaí; e o Nginx instalado na máquina atende outros sites, sem participar do fluxo do Trocaí. Essa condição é indicada em nota no diagrama.

5 DIAGRAMA DE ARQUITETURA DEVOPS

O diagrama de arquitetura DevOps (Figura 3, Apêndice C) apresenta as ferramentas utilizadas em cada etapa do ciclo de vida do software e o fluxo que leva uma alteração de código da máquina do desenvolvedor até o ambiente de produção, tal como está implementado hoje. O diagrama está organizado em quatro raias (Desenvolvimento, Versionamento e CI/CD, Deploy, e Operação e Monitoramento) e possui legenda para as cores e estilos das linhas. Elementos desenhados em cinza tracejado representam lacunas identificadas, ainda não implementadas.

Na etapa de desenvolvimento, cada integrante executa localmente o backend (Python e Django), o frontend (Next.js, com npm) e o banco de dados PostgreSQL em Docker Compose, utilizando o mesmo arquivo `docker-compose.yml` da produção, o que garante paridade de versões entre os ambientes. Nessa etapa também são executados, de forma manual, os 49 testes automatizados do backend e os scripts de lint (ESLint) e verificação de tipos (`tsc --noEmit`) do frontend.

O fluxo automatizado é numerado no diagrama da seguinte forma:

- 1) o desenvolvedor envia sua branch de trabalho ao repositório e abre um Pull Request, que é revisado e integrado à branch main;
- 2) o push resultante na branch main dispara o workflow `deploy.yml` no GitHub Actions, composto por um único job (deploy) que recebe as credenciais da VPS por meio dos GitHub Secrets;
- 3) no step de deploy, o runner conecta-se à VPS via SSH, sincroniza o código com `git reset --hard origin/main` e executa o script `infra/deploy.sh`, que instala as dependências, realiza o backup do banco com `pg_dump`, aplica as migrações, coleta os arquivos estáticos e compila o frontend;

- 4) somente após o build bem-sucedido os serviços trocai-api e trocai-web são reiniciados pelo systemd;
- 5) no step seguinte do mesmo job, os serviços são verificados com `systemctl is-active` e com requisições HTTP locais (`curl`);
- 6) o resultado do workflow é registrado no commit e, em caso de falha, o desenvolvedor é notificado pelo GitHub.

A análise do pipeline evidencia três limitações, registradas no diagrama e no `infra/README.md`. Primeiro, nenhum teste é executado no GitHub Actions: os testes, o lint e a verificação de tipos existem, mas rodam apenas localmente, de modo que um push na main segue diretamente para a produção. O diagrama representa como planejado um job de CI, anterior ao deploy (`needs: ci`), para executar essas verificações.

Segundo, a verificação de saúde não é totalmente confiável: os comandos utilizam a construção `cmd || (echo ... && exit 1)`, em que o `exit` encerra apenas o subshell criado pelos parênteses; como esse step não utiliza `set -e` nem `script_stop`, ele pode terminar com sucesso mesmo com um serviço inativo. Terceiro, a execução do build antes dos restarts evita que os serviços sejam reiniciados com um frontend quebrado, mas não constitui rollback: quando o build falha, as migrações já foram aplicadas, deixando o banco à frente do código, e o diretório `.next/` já está sendo reescrito enquanto o serviço anterior permanece no ar. Nesses casos, a recuperação é manual, a partir do backup gerado pelo `pg_dump`.

Na etapa de operação, a disponibilidade é apoiada pela diretiva `Restart=always` do systemd, enquanto os logs (`journald` e arquivos do Gunicorn) e os backups rotacionados fornecem os meios de diagnóstico e recuperação. Não há, contudo, monitoramento externo de disponibilidade: uma queda do sistema só é percebida manualmente, lacuna que o diagrama indica com a sugestão de um monitor HTTP periódico com alerta por e-mail.

6 DESCRIÇÃO E JUSTIFICATIVA DA INFRAESTRUTURA DE DEPLOY

6.1 Considerações iniciais

Esta seção descreve a infraestrutura adotada para a implantação da aplicação e apresenta a justificativa técnica de cada componente selecionado. As decisões foram orientadas por três critérios: a ausência de orçamento, característica de um projeto de natureza acadêmica; a adequação ao estágio atual de desenvolvimento; e a manutenção do menor grau possível de complexidade operacional.

Quanto à classificação, a solução adotada é de cloud, no modelo de infraestrutura como serviço (IaaS): a Oracle administra o hardware, a rede física e a virtualização, enquanto a equipe administra o sistema operacional, os serviços, o banco de dados e a aplicação. Trata-se, portanto, de uma aplicação autogerenciada sobre infraestrutura de nuvem, e não de self-hosting em sentido estrito, que exigiria que a equipe também fosse responsável pelo equipamento físico, pela energia, pela conectividade e pela segurança física do servidor. As duas alternativas são comparadas na Seção 6.2.

O Quadro 2 sintetiza as tecnologias empregadas em cada camada da infraestrutura.

Quadro 2 — Componentes da infraestrutura de implantação

Componente	Tecnologia
Hospedagem	Máquina virtual Ubuntu 24.04 — Oracle Cloud Always Free (cloud IaaS, administrada pela equipe)
Exposição pública	Cloudflare Tunnel
Banco de dados	PostgreSQL 18 em container Docker
Gerenciamento de processos	systemd
Integração e entrega contínuas	GitHub Actions

Fonte: Elaboração própria (2026).

As subseções seguintes discutem, para cada componente, as alternativas consideradas e os motivos que levaram à opção adotada.

6.2 Hospedagem: máquina virtual em cloud IaaS

Optou-se por hospedar a aplicação em uma máquina virtual administrada pela própria equipe, em detrimento de plataformas gerenciadas. A decisão decorre da restrição orçamentária do projeto e da necessidade de executar, de forma simultânea e sem degradação de desempenho, o backend, o frontend e o banco de dados em um mesmo ambiente.

Entre os provedores avaliados, o Oracle Cloud foi selecionado por oferecer, em sua camada gratuita permanente (Always Free), instâncias Ampere A1 de arquitetura ARM sem prazo de expiração. Em 15 de junho de 2026, a Oracle reduziu esse limite de 4 OCPUs e 24 GB de RAM para 2 OCPUs e 12 GB de RAM por tenancy (INFOQ, 2026; ORACLE, 2026), e a instância do projeto opera dentro do novo limite. O Quadro 3 apresenta o comparativo entre as ofertas gratuitas dos principais provedores considerados e a alternativa de servidor próprio; os dados referem-se às condições vigentes no momento da elaboração deste documento.

Quadro 3 — Comparativo das alternativas de hospedagem sem custo

Alternativa	Oferta gratuita relevante	Limitação identificada
Oracle Cloud (IaaS)	Ampere A1: 2 OCPUs e 12 GB de RAM, permanente (limite reduzido de 4 OCPUs e 24 GB em 15/06/2026)	Exige cadastro; arquitetura ARM; instâncias ociosas podem ser recuperadas (ver Seção 6.8)
AWS Free Tier	Plano gratuito com até 200 USD em créditos	Expira em 6 meses ou ao esgotar os créditos (contas criadas a partir de 15/07/2025)
GCP Free Tier	e2-micro (fração de vCPU, 1 GB), restrita a regiões us-*	Insuficiente para o processo de build do Next.js
Azure Free	B1s (1 vCPU, 1 GB)	Expira em 12 meses
Heroku	Camada gratuita descontinuada em 2022	Serviço pago
Railway	Crédito único de 5 USD no período de avaliação (30 dias)	Uso contínuo exige plano pago
Render	Serviço web gratuito suspenso após período de inatividade	Comportamento inadequado para ambiente de produção
Vercel + Supabase	Gratuito, com limites por projeto	Dependência de fornecedor; banco de dados em serviço distinto
Servidor próprio (self-host)	Computador da equipe ou do laboratório da universidade, sem custo de provedor	Depende de energia e internet locais, IP dinâmico ou CGNAT; sem SLA; hardware, backup e segurança física por conta da equipe

Fonte: Elaboração própria (2026), com base em AWS (2025), Google Cloud ([s.d.]), Heroku (2022), Microsoft ([s.d.]), Oracle (2026), Railway ([s.d.]), Render ([s.d.]), Supabase ([s.d.]) e Vercel ([s.d.]).

Verifica-se que, mesmo após a redução de junho de 2026, nenhum dos demais provedores disponibiliza recursos comparáveis sem custo e sem prazo de expiração: os 12 GB de RAM da Oracle equivalem a doze vezes a memória das instâncias gratuitas da AWS, do GCP e do Azure. Esse resultado justifica a escolha do Oracle Cloud.

6.2.1 Plataformas como serviço (PaaS)

Plataformas como Heroku, Railway e Render abstraem a administração da máquina, porém impõem restrições de execução relevantes, entre as quais o cold start, a suspensão por inatividade, o limite de memória durante o build e a cobrança proporcional ao número de processos. Considerando que apenas o build do Next.js consome aproximadamente 500 MB de memória RAM, tais limitações tornariam o processo inviável ou excessivamente lento nas camadas gratuitas dessas plataformas. Na máquina virtual adotada, os 12 GB disponíveis asseguram margem suficiente para a execução concorrente do backend, do frontend e do banco de dados.

6.2.2 Arquiteturas serverless

A hipótese de hospedar o frontend em uma plataforma serverless, como a Vercel, mantendo o backend em serviço distinto, foi descartada por três razões. A distribuição dos componentes entre provedores diferentes introduz latência de rede na comunicação entre eles, duplica a gestão de segredos e credenciais e, sobretudo, exigiria adaptações significativas no backend, uma vez que a execução do Django em modelo serverless depende de ferramentas adicionais e de ajustes no gerenciamento de conexões com o banco de dados. Para o estágio atual do projeto, a abordagem monolítica em uma única máquina mostra-se mais simples e mais previsível.

6.2.3 Servidor próprio (self-host)

A alternativa de self-hosting em sentido estrito, com um computador da equipe ou de um laboratório da universidade, também eliminaria o custo de provedor e daria controle total sobre o hardware. Entretanto, a disponibilidade do sistema passaria a depender da energia elétrica e da conexão de internet locais, geralmente sem IP fixo e frequentemente atrás de CGNAT, e a equipe assumiria a manutenção do equipamento, os backups externos e a segurança física. Para um sistema de extensão que deve permanecer acessível à comunidade, a máquina virtual em cloud oferece maior disponibilidade com o mesmo custo nulo, razão pela qual foi preferida.

6.3 Exposição pública: Cloudflare Tunnel

A alternativa convencional para expor a aplicação consistiria em instalar o Nginx na VPS como proxy reverso, abrir as portas 80 e 443 no firewall e configurar certificados TLS por meio do Let's Encrypt. Optou-se, em seu lugar, pelo Cloudflare Tunnel (CLOUDFLARE, [s.d.]), com base em três vantagens concretas:

- **Ausência de portas HTTP/HTTPS expostas.** O agente cloudflared estabelece uma conexão de saída em direção à borda do Cloudflare, de modo que as portas 80 e 443 permanecem fechadas no firewall da VPS. A única porta liberada é a 22, utilizada pelo pipeline de deploy via SSH com autenticação por chave.
- **Gestão de TLS e mitigação de DDoS sem custo.** O Cloudflare realiza a terminação TLS, fornece o certificado e absorve ataques volumétricos, sem necessidade de configuração adicional por parte da equipe.

- **Ocultação do endereço IP de origem.** Como o tráfego da aplicação passa pelo Cloudflare, o IP real da instância não é divulgado pelo DNS, o que dificulta ataques direcionados à máquina.

O custo da solução é nulo para o volume de uso atual. Como a VPS é compartilhada, o túnel também atende outros projetos, e o Trocaí possui regras de ingress próprias para os seus dois hostnames; a configuração de exemplo está versionada em `infra/cloudflared/`.

6.4 Banco de dados: PostgreSQL em container Docker

Serviços de banco de dados gerenciado, como AWS RDS, GCP Cloud SQL, Supabase e Neon, apresentam custo a partir de 10 a 25 USD mensais em suas menores instâncias, ou disponibilizam camadas gratuitas com restrições severas de armazenamento e de conexões simultâneas. Diante disso, optou-se pela execução do PostgreSQL 18 em container Docker na própria VPS, decisão que apresenta as seguintes vantagens para o projeto:

- O mesmo arquivo `docker-compose.yml` é utilizado em desenvolvimento e em produção, o que assegura paridade de versão entre os dois ambientes; a porta do host é parametrizada (7510 em produção, pois a 5432 pertence a outro projeto da mesma VPS).
- O bind do serviço em 127.0.0.1 restringe o acesso ao banco à própria máquina, condição equivalente ao isolamento de rede oferecido por um banco gerenciado.
- Os dados ficam no volume `postgres_data`, que persiste independentemente do ciclo de vida do container.
- O script `infra/deploy.sh` executa `pg_dump` automaticamente antes de cada migração, o que provê rotina de backup sem custo adicional.

Reconhece-se como limitação da solução a inexistência de réplica de leitura e de mecanismo de failover automático. Tal restrição é considerada aceitável para um MVP de caráter acadêmico, mas deveria ser reavaliada em um cenário de produção efetiva.

6.5 Gerenciamento de processos: systemd

O empacotamento do Django e do Next.js em containers conferiria maior portabilidade à aplicação, porém acrescentaria uma camada de indireção e ampliaria a complexidade do processo de depuração, em um momento em que a equipe ainda se dedica à construção da aplicação em si. Optou-se, portanto, pelo systemd, gerenciador de processos nativo do Ubuntu, cujas características atendem às necessidades do projeto: reinicialização

automática dos serviços em caso de falha, por meio da diretiva `Restart=always`; integração nativa com o `journald` para registro de logs; e suporte ao comando `sudo systemctl restart`, invocado pelo script de deploy. As units dos serviços estão versionadas em `infra/systemd/`.

6.6 Integração e entrega contínuas: GitHub Actions

O repositório do projeto encontra-se hospedado publicamente no GitHub, e o GitHub Actions é oferecido gratuitamente para repositórios públicos. O workflow implementado utiliza exclusivamente SSH e comandos de shell, dispensando runners customizados, registro de imagens Docker e armazenamento de artefatos. Alternativas como GitLab CI, Jenkins e CircleCI implicariam a migração do repositório ou a manutenção de infraestrutura adicional, sem benefício correspondente para o projeto. O mesmo serviço comporta, sem custo, o job de testes planejado descrito na Seção 5.

6.7 Ativação das contas e da estrutura

Para viabilizar a publicação, as contas e os recursos a seguir foram ativados e configurados:

- **Oracle Cloud:** conta criada na camada Always Free, instância Ampere A1 provisionada com Ubuntu 24.04 e configurada dentro do limite de 2 OCPUs e 12 GB, e Security List configurada para liberar apenas a porta 22 (SSH).
- **VPS:** Docker Engine instalado, container trocai-db em execução, diretórios `/opt/Trocai` e `/var/backups/trocai` criados e serviços trocai-api, trocai-web e cloudflared registrados no systemd, conforme o guia do `infra/README.md`.
- **Cloudflare:** domínio gerenciado pelo Cloudflare e túnel configurado com as rotas trocai.<domínio> e api-trocai.<domínio> apontando para as portas locais 3002 e 8000.
- **GitHub:** repositório público com GitHub Actions habilitado e GitHub Secrets `VPS_HOST`, `VPS_USER` e `VPS_SSH_KEY` cadastrados.

6.8 Riscos e limitações conhecidos

A escolha da infraestrutura envolve riscos que a equipe reconhece e acompanha:

- **Recuperação de instâncias ociosas.** A Oracle pode recuperar instâncias Always Free consideradas ociosas, isto é, quando, em um período de 7 dias, o uso de CPU (percentil 95), o uso de rede e o uso de memória ficam abaixo de 20% (ORACLE, 2026). Um sistema acadêmico com poucos acessos é candidato a esse critério. A

mitigação conhecida é converter a conta para Pay As You Go, modalidade em que os recursos dentro dos limites gratuitos não são cobrados e não estão sujeitos à recuperação; antes da conversão, é indispensável manter a instância dentro de 2 OCPUs e 12 GB, pois o excedente passa a ser cobrado.

- **Mudança unilateral de condições.** A redução de junho de 2026 ocorreu sem anúncio público (INFOQ, 2026), o que evidencia a dependência de um único fornecedor. A documentação de provisionamento em `infra/README.md` reduz o esforço de migração para outro provedor, caso necessário.
- **VPS compartilhada.** A mesma máquina hospeda outros projetos, de modo que uma sobrecarga ou falha em um deles pode afetar o Trocaí.
- **Pipeline sem testes e sem rollback.** Conforme descrito na Seção 5, os testes ainda não rodam no CI, a verificação de saúde pode não detectar um serviço inativo e uma falha de build após as migrações exige recuperação manual.

6.9 Síntese da justificativa

O conjunto formado por máquina virtual gratuita em cloud IaaS, Cloudflare Tunnel, containerização restrita ao banco de dados e gerenciamento de processos via `systemd` constitui uma pilha tecnológica que maximiza o aproveitamento dos recursos disponíveis sem custo, mantém baixa a complexidade operacional e preserva o controle integral sobre o ambiente de execução. Mesmo com a redução recente da camada gratuita da Oracle, essas características mostram-se adequadas a um MVP acadêmico que necessita operar em ambiente de produção real, sem custo associado e com nível de confiabilidade suficiente para fins de demonstração, desde que os riscos listados na Seção 6.8 sejam acompanhados.

REFERÊNCIAS

ALLIED MARKET RESEARCH. **Do-It-Yourself (DIY) Home Improvement Retailing Market Size and Forecast, 2022–2030**. 2022. Disponível em: <<https://www.alliedmarketresearch.com/press-release/do-it-yourself-diy-home-improvement-retailing-market.html>>. Acesso em: 25 out. 2025.

AMAZON WEB SERVICES (AWS). **AWS Free Tier now offers \$200 in credits and 6-month free plan to explore AWS at no cost**. 16 jul. 2025. Disponível em: <<https://aws.amazon.com/about-aws/whats-new/2025/07/aws-free-tier-credits-month-free-plan/>>. Acesso em: 27 set. 2026.

CLOUDFLARE. **Cloudflare Tunnel**. Documentação. [s.d.]. Disponível em: <<https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>>. Acesso em: 27 set. 2026.

GOOGLE CLOUD. **Free cloud features and trial offer**. [s.d.]. Disponível em: <<https://cloud.google.com/free/docs/free-cloud-features>>. Acesso em: 27 set. 2026.

HEROKU. **Heroku's Next Chapter**. 25 ago. 2022. Disponível em: <<https://blog.heroku.com/next-chapter>>. Acesso em: 27 set. 2026.

INFOQ. **Oracle Quietly Halves Free Tier Ampere A1 Compute Limits with No Public Announcement**. jul. 2026. Disponível em: <<https://www.infoq.com/news/2026/07/oracle-cloud-free-tier-limits/>>. Acesso em: 27 set. 2026.

MICROSOFT. **Serviços gratuitos do Azure**. [s.d.]. Disponível em: <<https://azure.microsoft.com/pt-br/pricing/free-services>>. Acesso em: 27 set. 2026.

ORACLE. **Always Free Resources**. Oracle Cloud Infrastructure Documentation. 2026. Disponível em: <<https://docs.oracle.com/en-us/iaas/Content/FreeTier/resourceref.htm>>. Acesso em: 27 set. 2026.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS (ONU). **Transformando nosso mundo: a Agenda 2030 para o desenvolvimento sustentável**. Nova Iorque: ONU, 2015. Disponível em: <<https://brasil.un.org/pt-br/sdgs>>. Acesso em: 25 out. 2025.

PROGRAMA DAS NAÇÕES UNIDAS PARA O DESENVOLVIMENTO (PNUD). **Objetivos de Desenvolvimento Sustentável (ODS)**. [s.d.]. Disponível em: <<https://www.undp.org/pt/brazil/ods>>. Acesso em: 25 out. 2025.

RAILWAY. **Pricing Plans**. Documentação. [s.d.]. Disponível em: <<https://docs.railway.com/pricing/plans>>. Acesso em: 27 set. 2026.

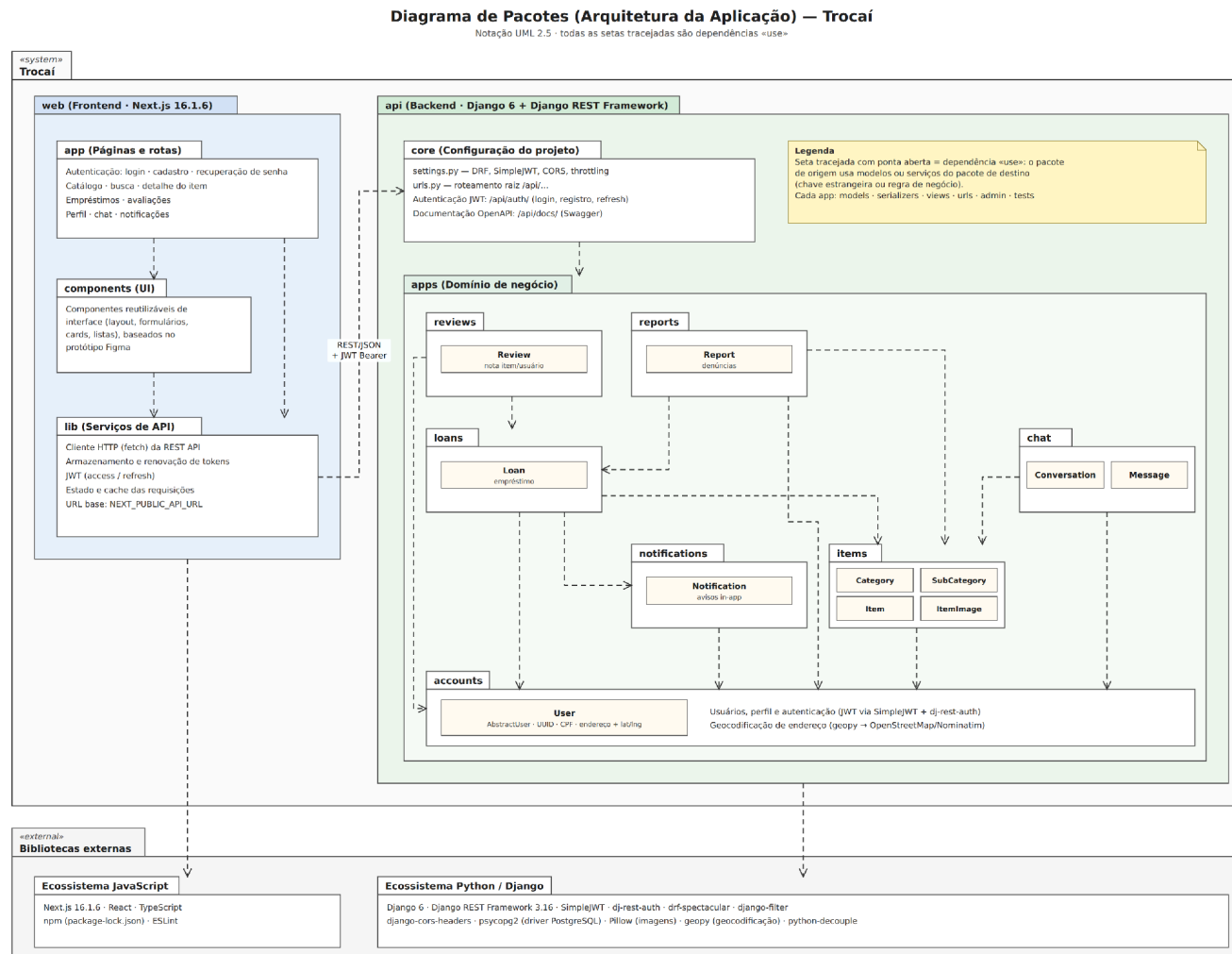
RENDER. **Deploy for Free**. Documentação. [s.d.]. Disponível em: <<https://render.com/docs/free>>. Acesso em: 27 set. 2026.

SUPABASE. **Pricing**. [s.d.]. Disponível em: <<https://supabase.com/pricing>>. Acesso em: 27 set. 2026.

VERCEL. **Pricing**. [s.d.]. Disponível em: <<https://vercel.com/pricing>>. Acesso em: 27 set. 2026.

APÊNDICE A — DIAGRAMA DE PACOTES UML (ARQUITETURA DA APLICAÇÃO)

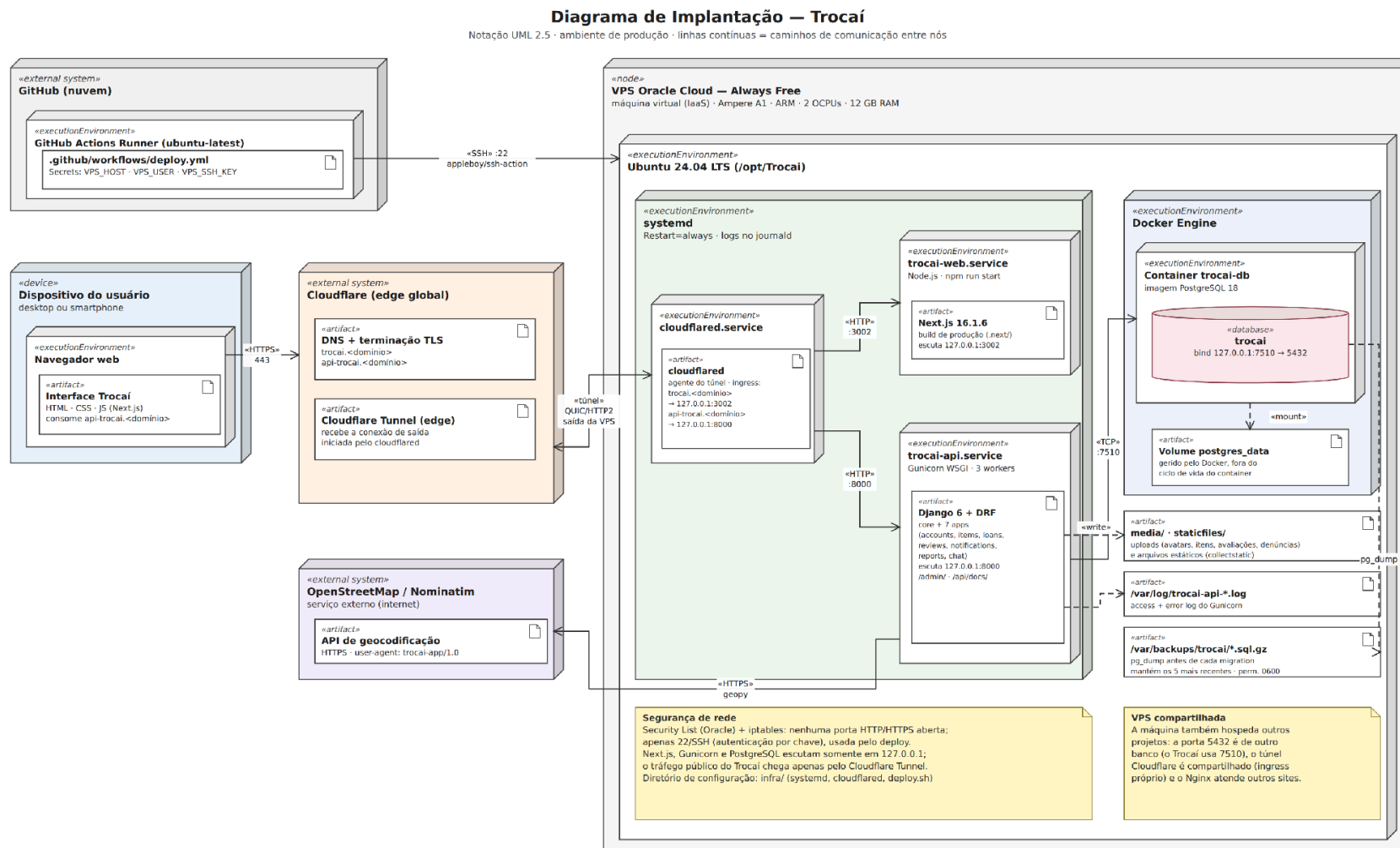
Figura 1 — Diagrama de pacotes UML da aplicação Trocaí



Fonte: Elaboração própria (2026).

APÊNDICE B — DIAGRAMA DE IMPLANTAÇÃO UML

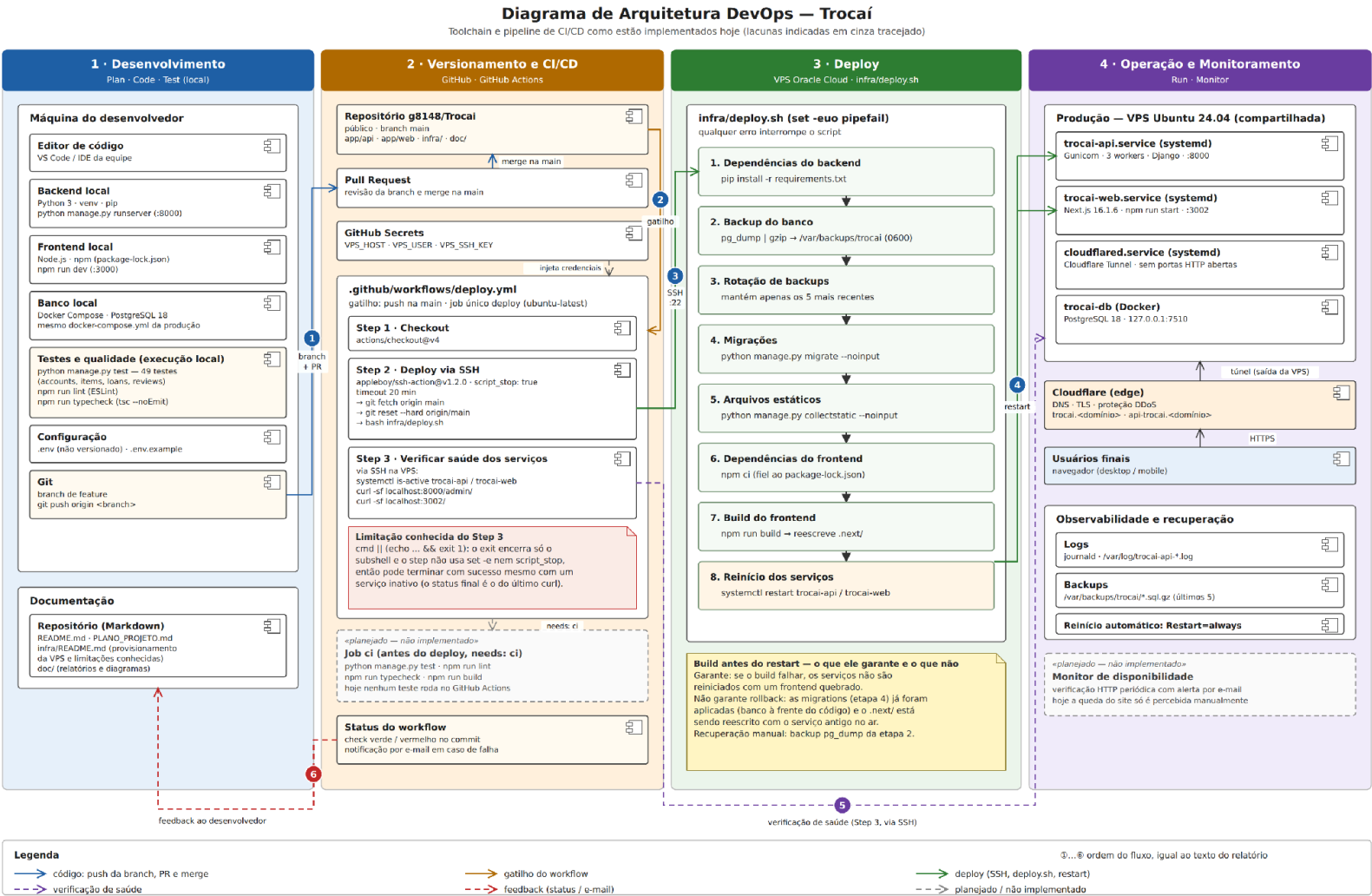
Figura 2 — Diagrama de implantação UML da aplicação Trocaí



Fonte: Elaboração própria (2026).

APÊNDICE C — DIAGRAMA DE ARQUITETURA DEVOPS

Figura 3 — Diagrama de arquitetura DevOps da aplicação Trocaí



Fonte: Elaboração própria (2026).